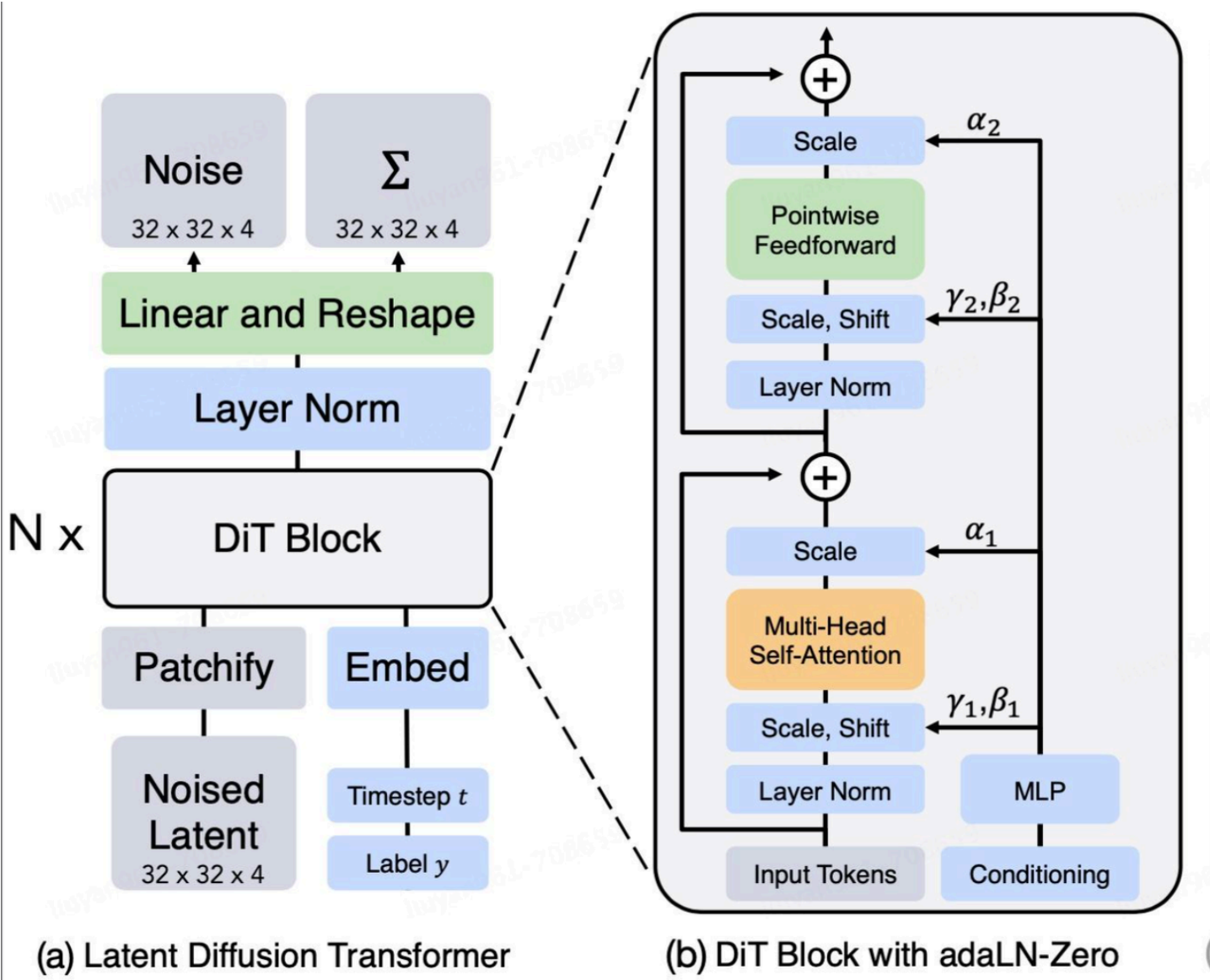


从零开始实现DiT

先看架构与原理：

其实DiT本质上还是依赖于ddpm的框架，只不过是把ddpm里面的UNet换成了ViT的变体，变成了DiT。先看DiT的核心架构图：



其中图b算得上是最核心的架构，它在这一层对时间特征进行提取，提取出6个参数，然后注入到LayerNorm之后进行放缩，相当于UNet里面注入了时间步的特征。值得注意的是这个 α_2 ，是要在一个MLP之后再注入。

这个 γ 的作用是scale，用于对线性层进行集体的放缩，而 β 的作用是shift，用于对权重进行一个偏置平移，然后 α 的作用是gate，用于与output的结果进行主元素相乘，控制输出对输入x的影响程度，再进行残差连接。

其实整体的公式应该是：

输入：x

运算是 $\text{attn}(x * (1 + \text{scale1}) + \text{shift1})$ 之后逐元素运用gate1得到x1, $x1 + x$

之后是 $\text{mlp}(x1 * (1 + \text{scale2}) + \text{shift2})$ 之后逐元素运用gate2得到x2, $x2 + x$ ，这就是这个block的输出

整个这张图本质上还是在预测噪声，输入噪声输出噪声。

现在来看代码：

先看时间编码：

这一步和transformer的位置编码是类似的，没有什么好说的

代码块

```
1  def timestep_embedding(t,dim,max_period=10000):
2      half = dim//2
3      freqs = torch.exp(
4          -
5          math.log(max_period)*torch.arange(0,half,device=t.device).float()/half
6      )
7      args = t.float()[None]*freqs[None]
8      emb = torch.cat([torch.cos(args),torch.sin(args)],dim=-1)
9      return emb
10
11 class TimeEmbedding(nn.Module):
12     def __init__(self,hidden_size,frequency_embedding_size=256):
13         super().__init__()
14         self.frequency_embedding_size = frequency_embedding_size#这个参数的含义
15         是什么？这个参数的含义是时间步嵌入的维度大小。它决定了我们在将时间步 t 转换为频率嵌入时，
16         得到的嵌入向量的维度。通常来说，频率嵌入的维度应该与模型中其他部分使用的维度相匹配，以便后
17         续的计算能够顺利进行。在这个代码中，频率嵌入的维度被设置为 256，但你可以根据需要进行调整。
18
19         self.hidden_size = hidden_size
20         self.mlp = nn.Sequential(
21             nn.Linear(frequency_embedding_size,hidden_size),
22             nn.SiLU(),
23             nn.Linear(hidden_size,hidden_size)
24         )
25
26     def forward(self, t):
27         t_freq = timestep_embedding(t,self.frequency_embedding_size)
```

然后是patch的位置编码，这一段是和ViT相同：

疑惑和解答全在注释里：

代码块

```

1  def get_1d_sincos_pos_embed(embed_dim, positions):
2      assert embed_dim % 2 == 0
3      omega = torch.arange(embed_dim//2).float()
4      omega = omega/(embed_dim/2)
5      omega = 1.0/(10000 ** omega)
6      out = positions.reshape(-1)[: , None]*omega#这个代码的作用是将输入的 positions
张量进行重塑，使其成为一个列向量。具体来说，positions.reshape(-1) 会将 positions 张量
展平为一个一维张量，而 [: , None] 则会在这个一维张量的基础上添加一个新的维度，使其成为一个
列向量。这样做的目的是为了后续的计算，因为我们需要将这个列向量与 omega 张量进行逐元素
相乘，以生成最终的正弦和余弦位置嵌入。
7      #逐元素相乘之后形状长什么样？假设 positions 的形状是 [P]，其中 P 是位置的数量，那
么 positions.reshape(-1)[: , None] 的形状将变为 [P, 1]。如果 embed_dim 是 D，那么
omega 的形状将是 [D//2]。当我们进行逐元素相乘时，out 的形状将是 [P, D//2]，因为每个位置
都会与 omega 中的每个频率进行相乘，生成一个新的向量。因此，最终 out 的形状是 [P,
D//2]。
8      return torch.cat([torch.sin(out), torch.cos(out)], dim=1)
9
10 def get_2d_sincos_pos_embd(embed_dim, grid_size):
11
12     assert embed_dim % 4 == 0
13     grid_h = torch.arange(grid_size).float()
14     grid_w = torch.arange(grid_size).float()
15     grid = torch.meshgrid(grid_h, grid_w, indexing='ij')
16     grid_h, grid_w = grid[0].reshape(-1), grid[1].reshape(-1)
17     emb_h = get_1d_sincos_pos_embed(embed_dim // 2, grid_h)
18     emb_w = get_1d_sincos_pos_embed(embed_dim // 2, grid_w)
19     return torch.cat([emb_h, emb_w], dim=1)
20 #emb_h 和 emb_w 的形状是什么？假设 grid_size 是 G，那么 grid_h 和 grid_w 的形状将是
[G*G]，因为我们将二维网格展平为一个一维张量。对于 embed_dim 是 D，那么 emb_h 和 emb_w
的形状将是 [G*G, D//2]，因为每个位置都会生成一个 D//2 维的嵌入向量。因此，最终返回的张量
的形状将是 [G*G, D]，因为我们将 emb_h 和 emb_w 沿着最后一个维度进行拼接。
21 #grid_h 和 grid_w 的形状是什么？假设 grid_size 是 G，那么 grid_h 和 grid_w 的形状将
是 [G, G]，因为 torch.meshgrid 会生成一个 GxG 的网格，其中 grid_h 包含了每个位置的行
索引，而 grid_w 包含了每个位置的列索引。之后，我们将它们展平为一维张量，所以最终 grid_h
和 grid_w 的形状将是 [G*G]。
22
23 class PatchEmbed(nn.Module):
24     def __init__(self, image_size, patch_size, in_channels=1, hidden_size=128):

```

```

25         super().__init__()
26         assert image_size%patch_size == 0
27         self.img_size = image_size
28         self.patch_size = patch_size
29         self.grid_size = image_size//patch_size
30         self.num_patch = self.grid_size**2
31
32         self.proj =
nn.Conv2d(in_channels,out_channels=hidden_size, kernel_size=patch_size, stride=pa
tch_size)
33
34     def forward(self,x):
35         x = self.proj(x)
36         x = x.flatten(2).transpose(1,2)
37
38         return x

```

再来看一下MLP层：

MLP没什么好说的吧：

代码块

```

1  class MLP(nn.Module):
2      def __init__(self, hidden_size, mlp_ratio = 4.0, drop = 0.0): #mlp_ratio 的含
        义是什么? mlp_ratio 的含义是多层感知机 (MLP) 中隐藏层的维度与输入维度的比例。具体来说，如
        果输入维度是 hidden_size，那么隐藏层的维度将是 hidden_size * mlp_ratio。这个参数控制
        了 MLP 中隐藏层的大小，通常来说，较大的 mlp_ratio 会增加模型的容量，但也可能导致过拟合。
        因此，在选择 mlp_ratio 时需要根据具体任务和数据集进行调整。
3          super().__init__()
4          mlp_hidden = int(hidden_size*mlp_ratio)
5          self.net = nn.Sequential(
6              nn.Linear(hidden_size,mlp_hidden),
7              nn.GELU(approximate='tanh'),
8              nn.Dropout(drop),
9              nn.Linear(mlp_hidden,hidden_size),
10             nn.Dropout(drop),
11         )
12
13     def forward(self,x):
14         return self.net(x)

```

来看一下图b的核心实现：

代码块

```

1  class DiTBlock(nn.Module):
2      def __init__(self, hidden_size, num_heads, mlp_ratio=4.0, drop=0.0):
3          super().__init__()
4
5          self.norm1 = nn.LayerNorm(hidden_size,eps=1e-
6,elementwise_affine=False)
6          self.attn =
nn.MultiheadAttention(hidden_size,num_heads,dropout=drop,batch_first= True)
7          self.norm2 = nn.LayerNorm(hidden_size,eps=1e-
8,elementwise_affine=False)
8          self.mlp = MLP(hidden_size,mlp_ratio,drop=drop)
9
10         self.adaLN_modulation = nn.Sequential(
11             nn.SiLU(),
12             nn.Linear(hidden_size,6*hidden_size),
13         )
14
15     def forward(self,x,c):
16         shift_msa, scale_msa, gate_msa, shift_mlp, scale_mlp, gate_mlp =
self.adaLN_modulation(c).chunk(6,dim=1)
17
18         attn_input = modulate(self.norm1(x),shift_msa,scale_msa)
19
20         '''modulate这个函数的作用是什么？
21         modulate 函数的作用是对输入张量 x 进行调制，具体来说，它会根据提供的 shift 和
scale 参数对 x 进行缩放和平移。
22         函数的实现是通过将 x 乘以 (1 + scale) 来进行缩放，然后再加上 shift 来进行平
移。
23         这个操作可以看作是一种自适应层归一化 (Adaptive Layer Normalization)，它允许
模型根据输入的条件 c 来动态调整特征表示，从而增强模型的表达能力和适应性。
24         在 DiTBlock 中，这种调制被应用于注意力机制和 MLP 的输入，以便更好地捕捉时间步信
息和图像特征之间的关系。'''
25
26         attn_output =
self.attn(attn_input,attn_input,attn_input,need_weight=False)
27
28         #这个attn的参数传的好奇怪啊，为什么？ 这个 attn 的参数传递方式是因为 PyTorch
的 MultiheadAttention 模块需要三个输入：查询 (query)、键 (key) 和值 (value)。在这个
代码中，attn_input 被同时用作查询、键和值，这是一种常见的自注意力机制的实现方式，称为“自
注意力” (self-attention)。通过将同一个输入作为查询、键和值，模型能够捕捉输入序列内部的
关系和依赖，从而更好地理解 and 处理输入数据。这种方式简化了代码，同时也符合自注意力机制的设计
原则。
29
30         x = x + gate_msa.unsqueeze(1)*attn_output
31
32         #这个gate是用来干什么的？ 这个 gate 的作用是控制注意力输出对输入 x 的影响程度。
通过将 gate_msa 扩展为与 attn_output 形状匹配的张量，并与 attn_output 逐元素相乘，我

```

们可以动态地调整注意力输出在最终结果中的权重。这种机制允许模型根据输入的条件 c 来灵活地增强或抑制注意力输出，从而提高模型的表达能力和适应性。

33 #我记得传统ViT是直接 $x = x + \text{attn_output}$ 的，这个 gate 是 DiTBlock 中引入的一个创新点，它为模型提供了更大的灵活性，使其能够根据不同的输入条件动态调整注意力输出的影响。这种设计可以帮助模型更好地捕捉时间步信息和图像特征之间的关系，从而提升模型在处理扩散模型任务时的性能。

```
34
35     mlp_input = modulate(self,self.norm2(x),shift_mlp,scale_mlp)
36     mlp_output = self.mlp(mlp_input)
37     mlp_output = x + mlp_output*gate_mlp.unsqueeze(1)
```

FinalLayer:

DiT最后一层不需要attention，也不需要引入gate，只需要做一个缩放和偏置就行。

代码块

```
1  class FinalLayer(nn.Module):
2      def __init__(self, hidden_size, patch_size, out_channels):
3          super().__init__()
4          self.final_norm = nn.LayerNorm(hidden_size,eps=1e-
5      6,elementwise_affine=False)
6          self.adaLN_modulation = nn.Sequential(
7              nn.SiLU(),
8              nn.Linear(hidden_size,2*hidden_size),
9          )
10         self.linear = nn.Linear(hidden_size,patch_size*patch_size*out_channels)
11
12     def forward(self,x,c):
13         shift,scale = self.adaLN_modulation(c).chunk(2,dim=-1)
14         x = modulate(self.final_norm(x),shift,scale)
15         return self.linear(x)
```

DiT的核心拼装:

代码块

```
1  class DiT(nn.Module):
2      def __init__(
3          self,
4          image_size,
5          patch_size,
6          in_channels,
7          hidden_size,
8          depth=4,
```

```

9         num_heads=4,
10        mlp_ratio=4.0,
11        drop=0.0
12    ):
13        super().__init__()
14        self.image_size = image_size
15        self.patch_size = patch_size
16        self.in_channels = in_channels
17        self.out_channels = in_channels
18        self.hidden_size = hidden_size
19
20        self.x_embedder =
PatchEmbed(image_size,patch_size,in_channels,hidden_size)
21        self.t_embedder = TimeEmbedding(hidden_size)
22        self.block = nn.ModuleList([
23            DiTBlock(hidden_size,num_heads,mlp_ratio,drop)
24            for _ in range(depth)
25        ])
26        self.final_layer = FinalLayer(hidden_size,patch_size,in_channels)
27
28        pos_embed =
get_2d_sincos_pos_embd(hidden_size,self.x_embedder.grid_size)
29
self.register_buffer('pos_embed',pos_embed.unsqueeze(0),persistent=False)
30
31    def unpatchify(self,x):
32        B,N,patch_dim = x.shape
33        p = self.patch_size
34        C = self.out_channels
35        grid = int(N**0.5)
36        x = x.reshape(B,grid,grid,p,p,C)
37        x = torch.einsum('nhwpqc->nchpwq',x)#这个函数是什么？这个函数是 PyTorch 中
的 einsum 函数，它用于执行爱因斯坦求和约定的张量操作。在这个代码中，'nhwpqc->nchpwq' 是
一个字符串，指定了输入张量 x 的维度标签以及输出张量的维度标签。具体来说，输入张量 x 的维度
被标记为 n（批次大小）、h（网格高度）、w（网格宽度）、p（补丁大小）、q（补丁大小）和 c
（通道数）。通过指定 'nhwpqc->nchpwq'，我们告诉 einsum 函数将输入张量的维度重新排列为 n
（批次大小）、c（通道数）、h（网格高度）、p（补丁大小）和 w（网格宽度）。这种操作可以看作
是对输入张量进行转置和重塑，以便将补丁重新组合成原始图像的形状。
38        imgs = x.reshape(B,C,grid*p,grid*p)#为什么要保留C？保留 C 是因为 C 代表了图
像的通道数，例如对于 RGB 图像，C 通常是 3。通过保留 C，我们能够确保在重塑过程中正确地处理
图像的颜色通道，从而得到正确的输出图像。最终输出的 imgs 张量将具有形状 [B, C, H, W]，其
中 H 和 W 是根据 grid 和 patch_size 计算得出的图像高度和宽度。这种结构使得模型能够生成
具有正确通道数的图像输出。
39        return imgs
40
41    def forward(self,x,t):
42        x = self.x_embedder(x) + self.pos_embed

```

小结：

总体敲下来感觉DiT的思路还是很简单的，是ddpm把UNet换成ViT，然后引入shift，scale和gate之后的结果。

其余的步骤完全照抄ddpm了。

写在最后：最近期末月了，报告篇幅变短变水了。。。确实时间也是紧张了哈哈，但是任务还是要完成，持续学习还是很有必要的。

另外GitHub上面还补了DiT和ddpm的关系说明~